

CIS 22 – INTRO TO PROGRAMMING

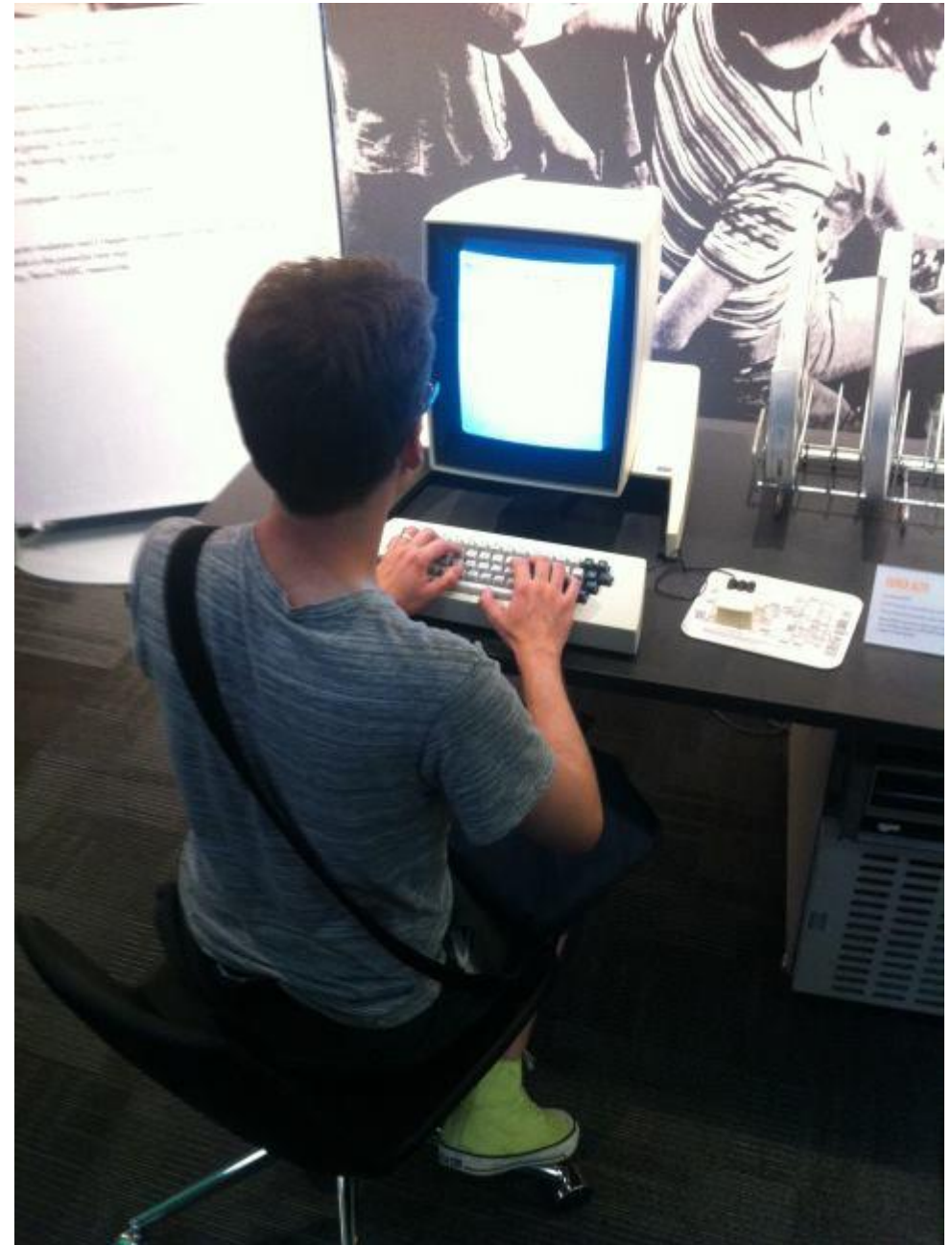
ABOUT THIS CLASS

practical details

- CIS 22: Intro to Programming
- NOTE: Despite the course name, This is NOT a good first programming class. There's going to be a bit of review at the front, but it'll ramp up real quick
- Meets Tuesdays and Thursdays in this room from 11:00 AM to 12:55 PM, typically with a 10 minute break toward the middle of the class.
- Although officially we have separate lab and lecture sections, generally Tuesday will be mostly lecture and Thursday mostly lab

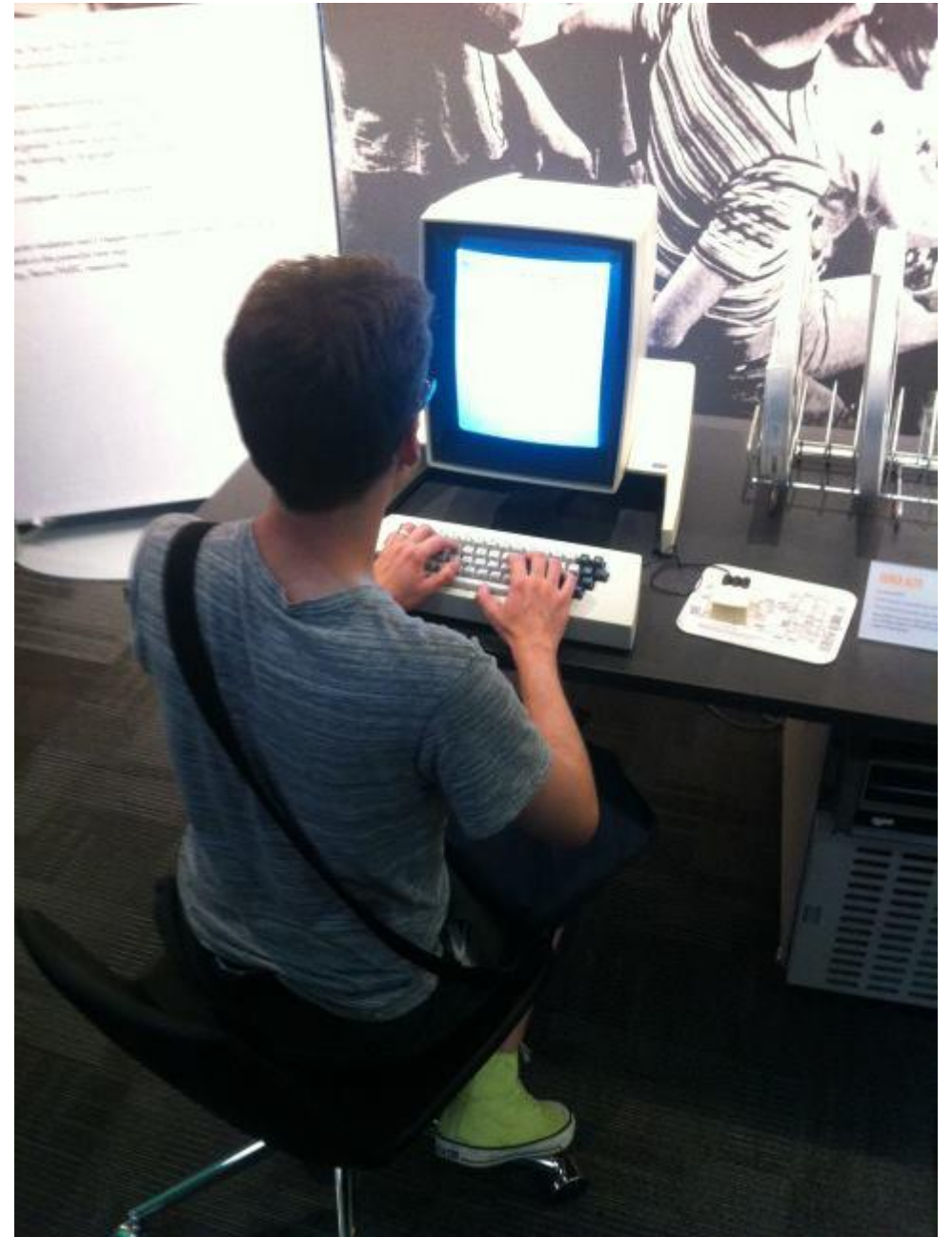
ABOUT ME

- Preferred pronouns: he/him
- My first degree was an associate's degree from Bellevue Community College
- A while ago I finished up a Master's in Computer Science at Stanford (I specialized in CS education and compiler design)



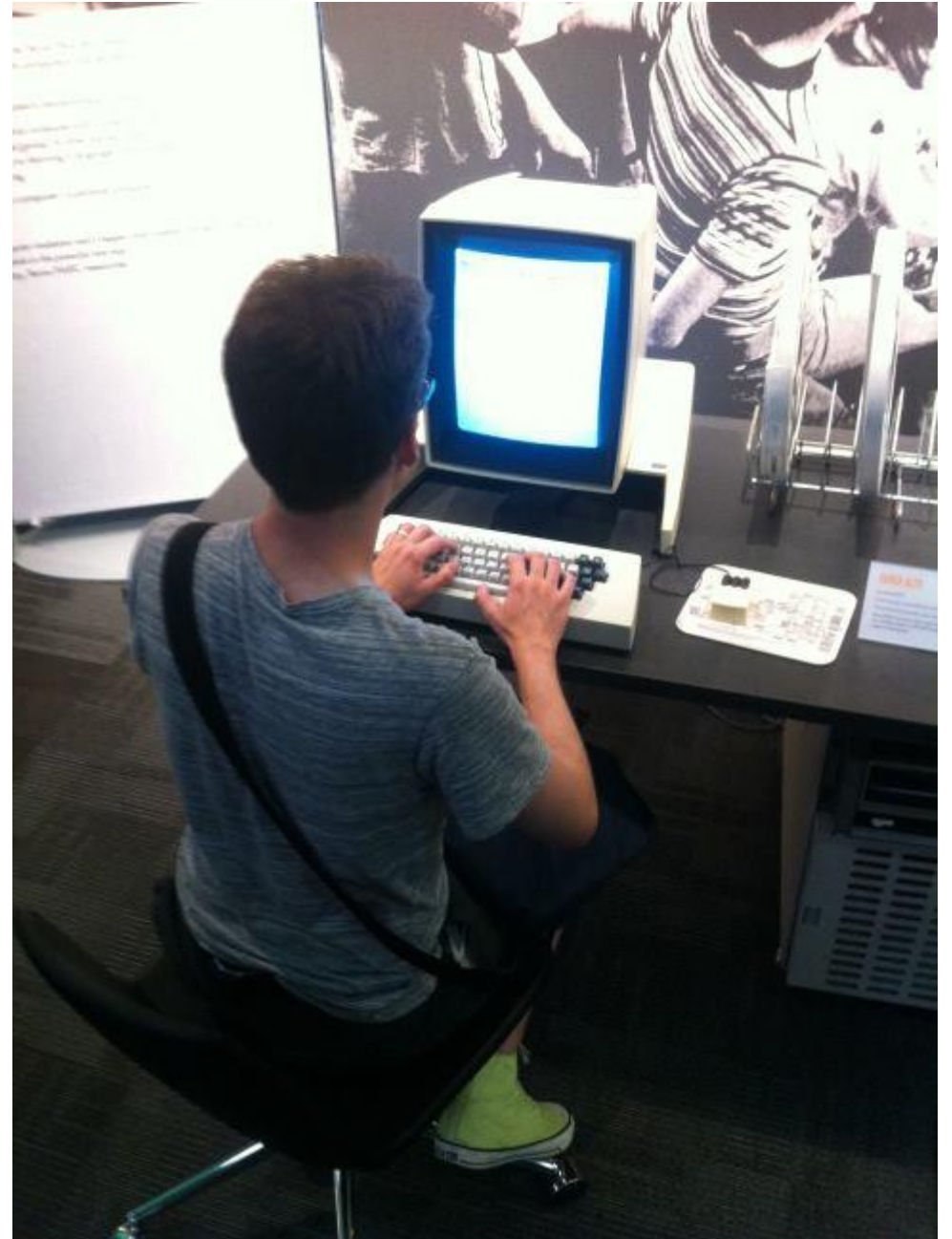
ABOUT ME

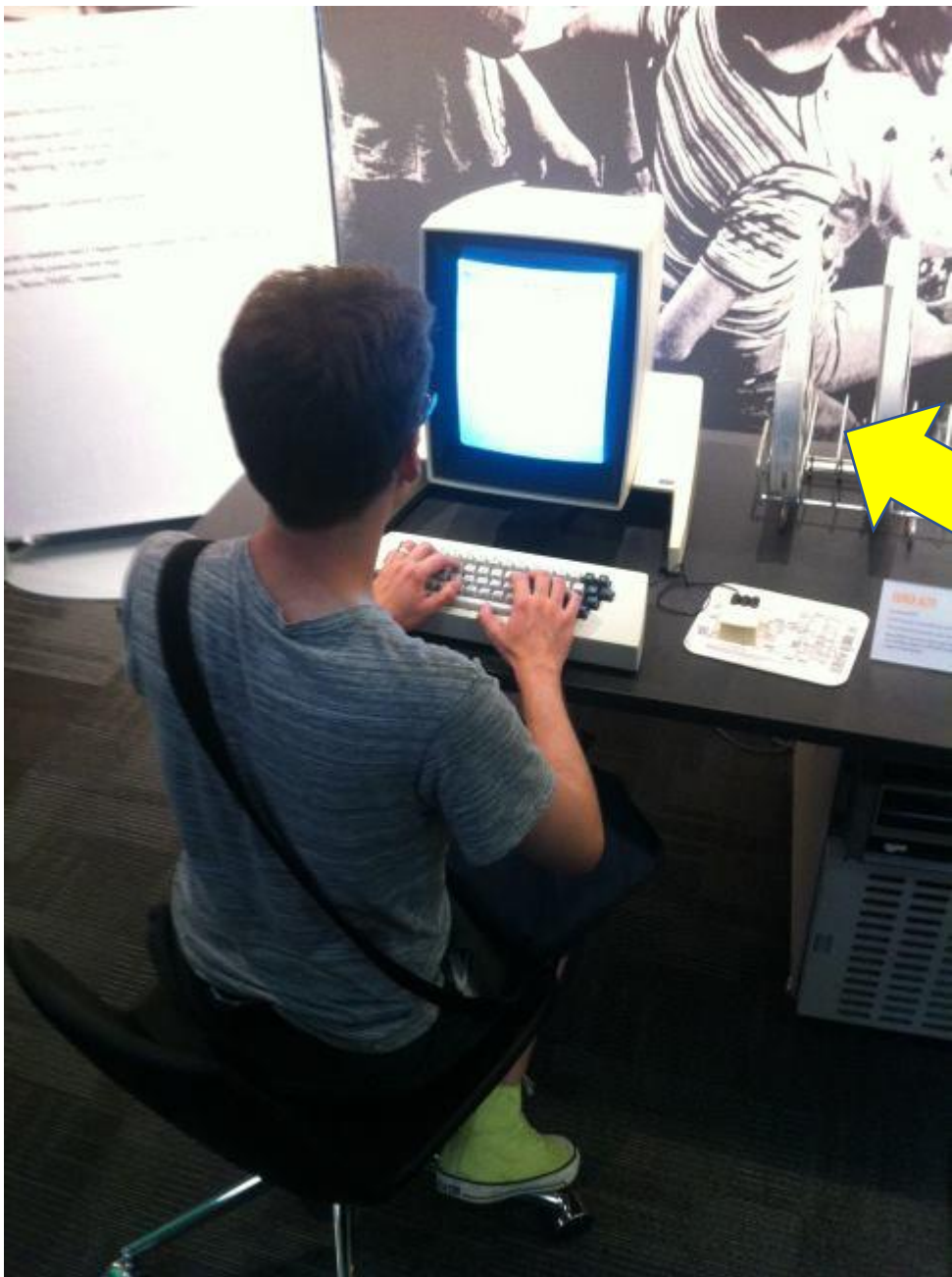
- Before that, I got a PhD in the history of computing (also from Stanford)
- (which is why I was so excited about using this computer)



ABOUT ME

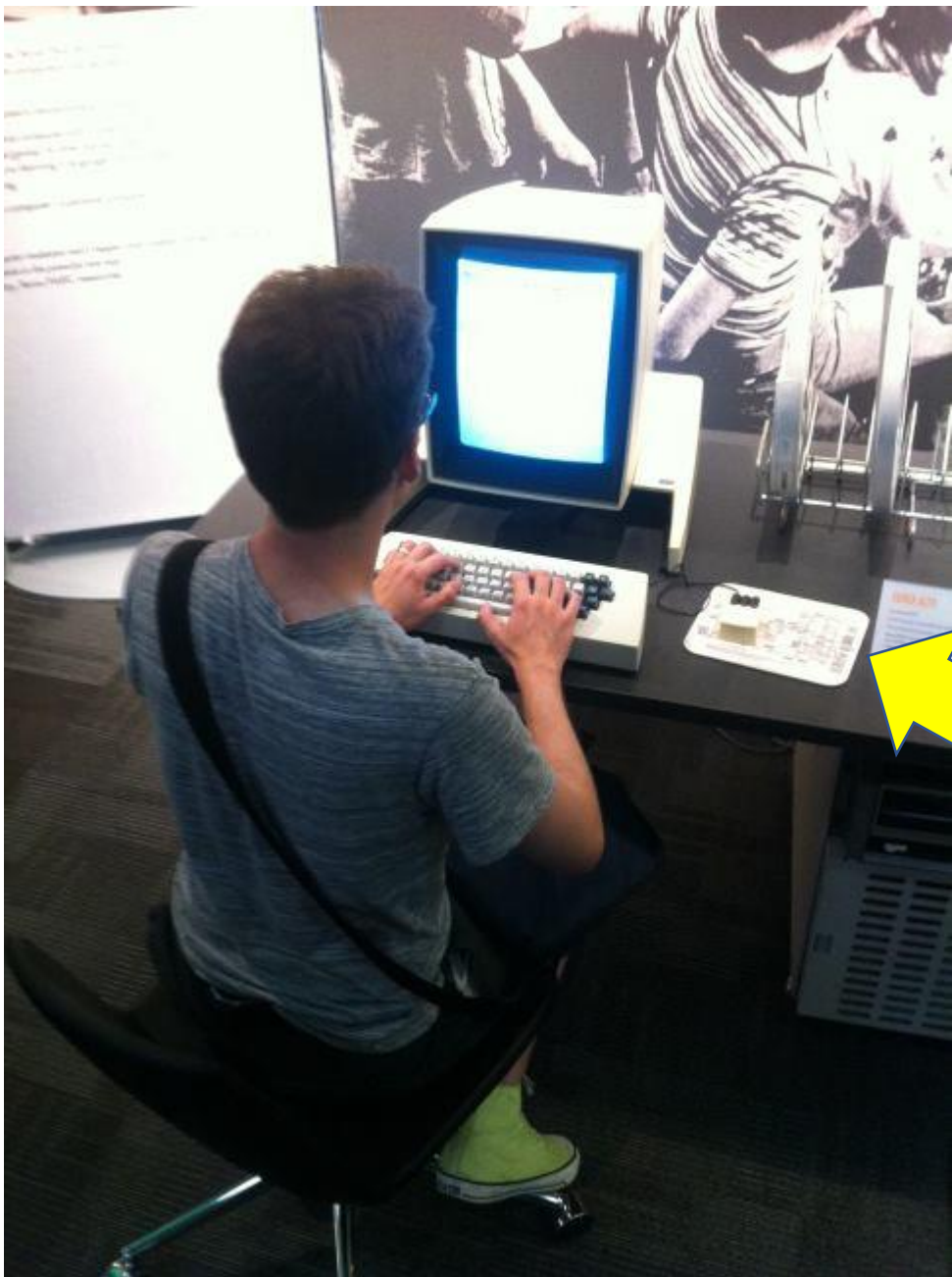
- (I'll sneak some material from the history of computing into class, but I promise to keep it under control)





ABOUT THIS PICTURE

- The machine I'm sitting at here is a Xerox Alto
- It was the first computer with a graphical user interface (windows, icons, pointer)
- First computer with a mouse!
- Built in 1973!



ABOUT THIS PICTURE

- The machine I'm sitting at here is a Xerox Alto
- It was the first computer with a graphical user interface (windows, icons, pointer)
- **First computer with a mouse!**
- Built in 1973!

ABOUT THIS CLASS: what you'll learn

- Here's what we all should know how to do by the end of the semester
- Design, implement, test, and debug programs that use conditional statements, loop structures, methods, and object-oriented concepts including classes, inheritance, arrays, virtual classes, templates, and exception handling for writing programs.

ABOUT THIS CLASS: What you should know coming in

You should have at least some experience with the following:

- Designing, implementing, testing, and debugging programs that use conditional statements, loop structures, and functions.
- Decomposing complex problems into relatively simple functions.
- Programs that use arrays.

ABOUT THIS CLASS: what you'll learn

- By the end of the semester, we should all be able to:
 - Design, implement, test, and debug programs that use classes, inheritance, arrays, virtual classes, templates, and exceptions.
 - Write programs that use pointers to implement the following data structures: linked lists, binary search trees, and heaps
 - Demonstrate an understanding of late binding and polymorphism.
 - Understand memory management in C++, including the distinction between stack-allocated memory and heap-allocated memory.
 - Demonstrate understanding of the importance of types, type incompatibility, and type-checking in providing abstraction and safety.
 - Produce implementations of breadth-first search, depth-first search, and other canonical computer science algorithms

ABOUT THIS CLASS: what you'll learn

- By the end of the semester, we should all be able to:
 - Design, implement, test, and debug programs that use classes, inheritance, arrays, virtual classes, templates, and exceptions.
 - Write programs that use pointers to implement the following data structures: linked lists, binary search trees, and heaps
 - Demonstrate an understanding of late binding and polymorphism.
 - Understand memory management in C++, including the distinction between stack-allocated memory and heap-allocated memory.
 - Demonstrate understanding of the importance of types, type incompatibility, and type-checking in providing abstraction and safety.
 - Produce implementations of breadth-first search, depth-first search, and other canonical computer science algorithms

This is the interesting stuff.
we're learning all this
syntax so that we can use it to
do real computer science.



ABOUT THIS CLASS: what you'll learn

- Demonstrate understanding of the importance of types, type incompatibility, and type-checking in providing abstraction and safety.



we're not just going to talk about how to do serious object-oriented programming in C++, but we're also going to get deep into the theory behind object oriented programming in general. basically you're going to understand how and why your compiler is helping you out when it gives you type errors.

... but before all that

- At the start of my CS classes, I like to do some exercises using a simple small language before we get to the large, complex languages we'll be working in for the rest of the semester.
- Why? We have to learn a lot of syntax to use a powerful language like C++, and sometimes that can obscure the fact that computer science is about *algorithm design* first and foremost.
- The exercises we'll do today will (hopefully) wake up the algorithm design parts of your brain a little bit.

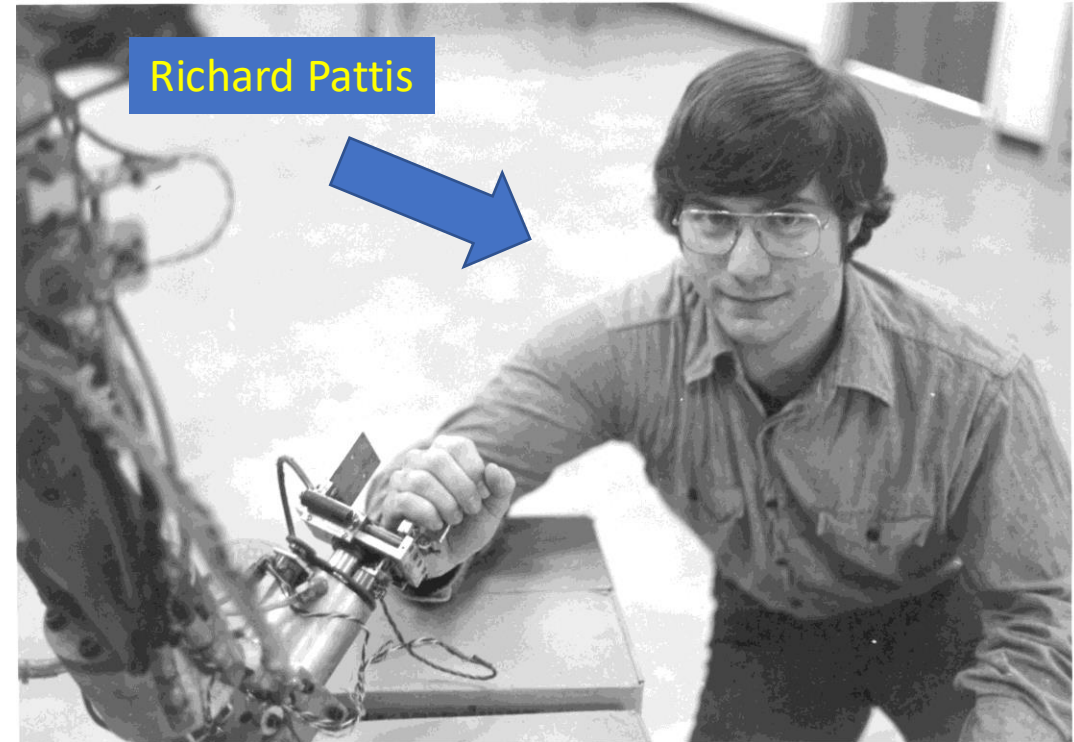


KAREL THE ROBOT

has escaped from Stanford

KAREL HISTORY

- Originally developed by a computer scientist named Richard Pattis while he was working on his master's degree at Stanford
- First released in 1981
- Karel has been used at Stanford ever since



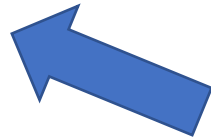
KAREL HISTORY

- Originally developed by a computer scientist named Richard Pattis while he was working on his master's degree at Stanford
- First released in 1981
- Karel has been used at Stanford ever since
- This class is the first at SCC to use Karel



KAREL HISTORY

- This class is the first at SCC to use Karel



(and the first to use the implementation of Karel I wrote over the summer, so there will likely be bugs in that implementation. Sorry about that!)

KAREL HISTORY

- This class is the first at SCC to use Karel



(my potentially buggy implementation lives here)

<https://ben-allen.github.io/CIS-022/karel-cpp.html>

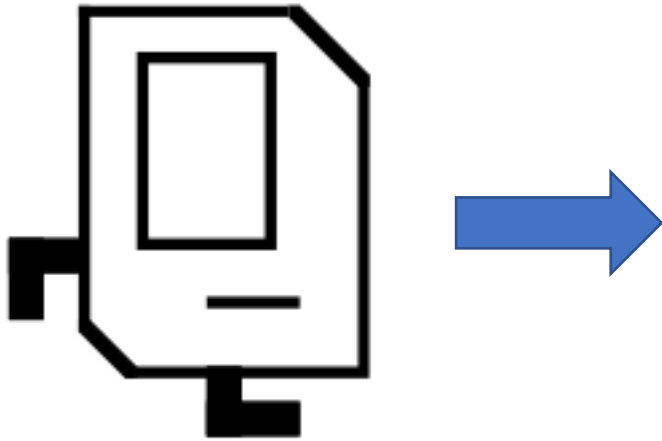
ABOUT KAREL

- Karel was born in 1981
- Karel looks a little bit like a 1980s Macintosh computer



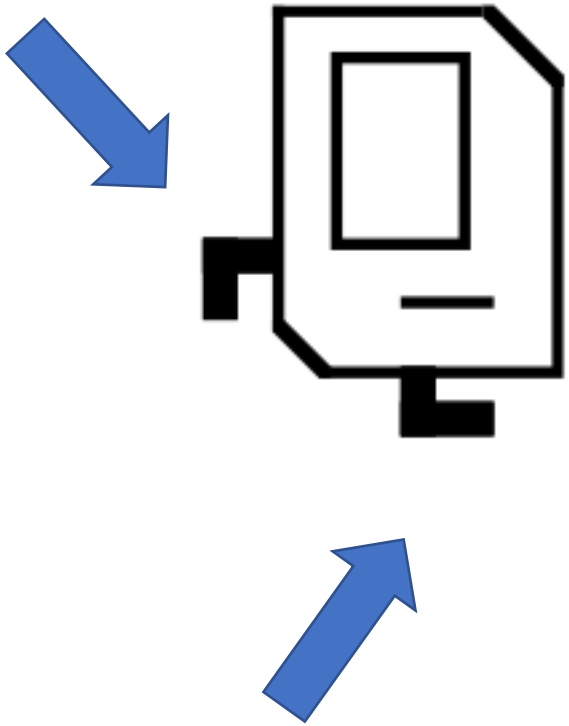
ABOUT KAREL

- Karel is facing RIGHT in this picture



ABOUT KAREL

- Karel is facing RIGHT in this picture
- These are Karel's feet

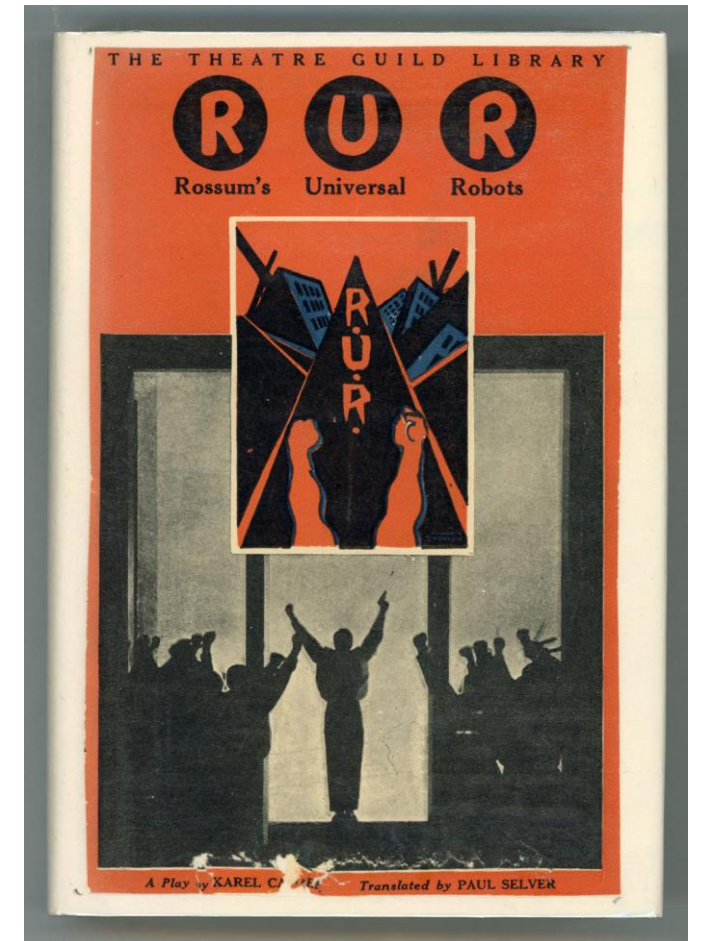


ABOUT KAREL



- Karel is named after Karel Čapek, the Czech playwright who invented the word "robot" back in 1920.

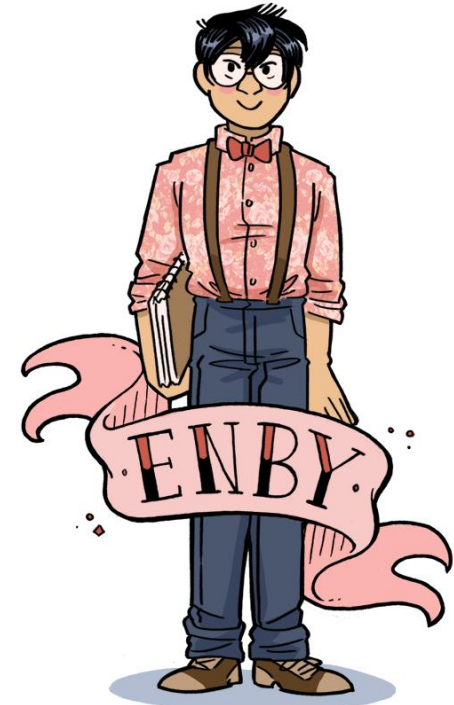
Rossum's Universal Robots. It's a pretty good play!



ABOUT KAREL



- Karel is a non-binary robot
- Karel uses she/her or they/their pronouns (either way is fine)



KAREL COMMANDS

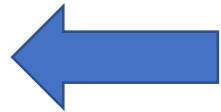
- Here are the commands you can give Karel:
 - `move();`
 - `turn_left();`
 - `put_beeper();`
 - `pick_beeper();`



These are all FUNCTION CALLS. how do you know? the parentheses after each of them.

WHERE DO COMMANDS GO?

```
int main() {
```



`int main()` is where program execution starts

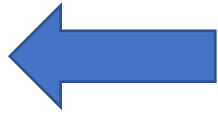
```
}
```

what does `int main()` mean? don't worry too hard about it yet. It's something called a *function*; `main` is the function where Karel looks for instructions

WHERE DO COMMANDS GO?

```
int main() {
```

Everything inside {



```
}
```

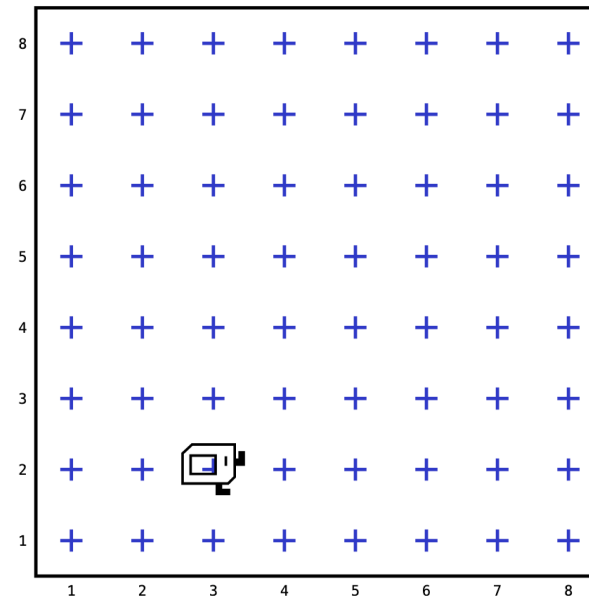
```
}
```

is executed step by step when your program runs

WHERE DO COMMANDS GO?

```
int main() {  
    move();  
    move();  
    turn_left();  
    move();  
  
    return 0;  
}
```

This program causes Karel to move forward twice, turn left, then move again



- If a command is by itself on a line, that's called a STATEMENT

```
move();  
turn_left();  
put_beeper();
```



each of these are statements

- Single-line statements end with a ;

```
move();  
turn_left();  
put_beeper();
```



If you forget the semicolon, your
program won't run and you will be sad 😞

- Karel also accepts a few COMPOUND STATEMENTS

```
while (front_is_clear())  
{  
}
```



this is a CONDITIONAL STATEMENT

this compound statement executes
only while this conditional statement is
TRUE

everything INSIDE the {

}

is part of the compound statement

(this is just like a while loop in C++, Java, or Python)

notice
that front_is_clear is a
FUNCTION – it's got parentheses after it



```
while (front_is_clear()){  
  
}
```

this is a CONDITIONAL STATEMENT

this compound statement executes
only while this conditional statement is
TRUE

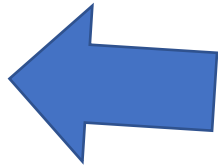


everything INSIDE the {

}

is part of the compound statement

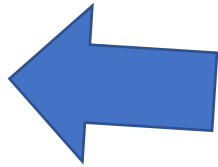
```
while (front_is_clear()){  
    move();  
    turn_left();  
    move();  
}
```



notice that the COMPOUND STATEMENT
contains SIMPLE STATEMENTS

each of the simple statements ends with a ;

```
while (front_is_clear()){  
    move();  
    turn_left();  
    move();  
}
```



notice that the COMPOUND STATEMENT
contains SIMPLE STATEMENTS

each of the simple statements ends with a ;

If Karel doesn't have something stopping them from moving, they'll move forward, turn left, then move forward again. Then they'll check if their front is clear again. If it is, they repeat everything inside the { } block

KAREL REFERENCE SHEET

KAREL REFERENCE SHEET:

These are the commands Karel knows out of the box.

```
move();  
turn_left();  
put_beeper();  
pick_beeper();
```

These are the conditional functions Karel knows.
These can be used in while loops and in if statements:

```
front_is_blocked()  
left_is_clear()  
left_is_blocked()  
right_is_clear()  
right_is_blocked()  
back_is_clear()  
back_is_blocked()  
beepers_present()  
no_beepers_present()
```

syntax highlighting



```
int main(){  
    // your code here...  
    while (front_is_clear()) {  
        move();  
    }  
    put_beeper();  
}
```

notice that some words are *highlighted* in the Karel interface

this indicates that they are *language keywords*

syntax highlighting helps you keep track of what parts of the program are which, and let you see the program's structure at a glance

all programmers' editors use syntax highlighting extensively

comments

```
int main(){  
    // your code here...  
    while (front_is_clear()) {  
        move();  
    }  
    put_beeper();  
}
```



Text after `//` is considered part of a *comment*, and is ignored by Karel

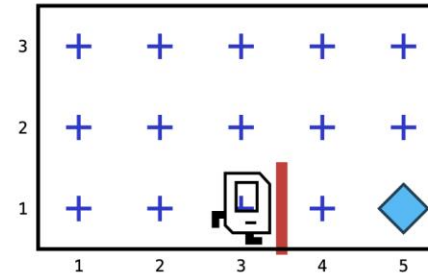
Comments are **very** useful for helping programmers remember what complex pieces of code do

crashing

- Today we'll discuss two ways of making Karel crash – if Karel crashes, that means your program is buggy. Completed programs *must not crash*

crashing – walking into a wall

```
int main() {  
    // your code here...  
    while (front_is_clear()) {  
        move();  
    }  
    move();  
}
```



► Run Program

Step

Pause

Reset

Speed medium

Show Text Descriptions

Line 8: Karel tried to move east from (3, 1) but a wall is in the way.

crashing – walking into a wall

Line 8: Karel tried to move east from (3, 1) but a wall is in the way

Whoops! Karel walked into a wall. If Karel tries to move into a wall, program execution stops, and cannot be resumed

Even if Karel has completed their task, crashing is considered wrong.

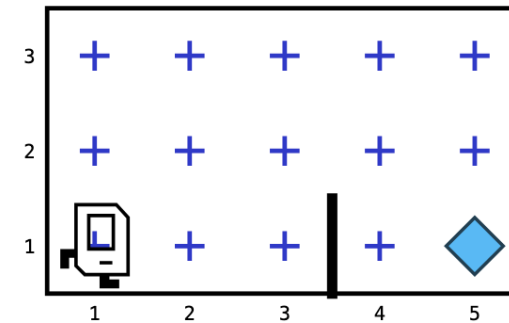
The interface will tell you on what line the crash occurred

crashing – picking up a beeper when there's no beeper present

```
#include "karel.h"
```

```
int main() {  
    pick_beeper();  
}
```

Around the wall



▶ Run Program

Step

Pause

Reset

Speed  medium

Show Text Descriptions

Line 4: Karel tried to pick up a beeper at (1, 1) but there isn't one there.

crashing – picking up a beeper when there's no beeper present

Line 5: Karel tried to pick up a beeper at (1, 1) but there isn't one there.

while loops only test *at the top of the loop*

- Check out this program. Does it crash when run on a 4x6 world?

```
#include "karel.h"
```

```
int main() {  
    while (front_is_clear()){  
        move();  
        move();  
    }  
  
}
```

while loops only test *at the top of the loop*

- Check out this program. Does it crash?

```
#include "karel.h"

int main() {
    while (front_is_clear()){
        move();
        move();
    }
}
```

YES! Why?

because Karel only
checks the front_is_clear()
condition **AT THE TOP OF THE
LOOP**

Once it enters the loop, it
doesn't check again until all
statements in the loop have
executed

while loops only test *at the top of the loop*

- Check out this program. Does it crash?

```
#include "karel.h"

int main() {
    while (front_is_clear()){
        move();
        move();
    }
}
```

YES! Why?

because Karel only checks the front_is_clear() condition **AT THE TOP OF THE LOOP**

Note: if you ran this on a board with an *odd number of columns*, the code would work. Bugs that happen *sometimes* are very annoying compared to bugs that happen *always*.

defining new functions

- Karel has one big limitation – **they can't turn right**
- You can fix that limitation by **defining your own function**
- You do this using the **void** keyword

defining new functions

- How do you turn right when you only know how to turn left?

defining new functions

- How do you turn right when you only know how to turn left?
- turn left three times!

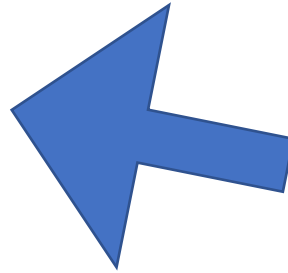
```
turn_left()
```

```
turn_left()
```

```
turn_left()
```

defining new functions

```
void turn_right(){  
    turn_left();  
    turn_left();  
    turn_left();  
}  
int main(){  
    turn_right();  
}
```



use the **void** keyword to define a new function

what does **void** mean? Don't worry too hard about it right away – it will make sense when we get to C++. Right now just think of it as "how to define a function"



once you define your new function, you can use it just like the built-in functions

if statements new functions

```
#include "karel.h"
```

```
void safe_move(){  
    if (front_is_clear()){  
        move();  
    }  
}
```

```
int main() {  
    safe_move();  
    safe_move();  
}
```

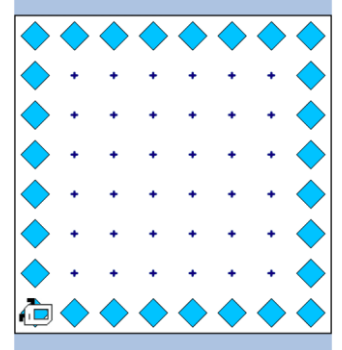
An if statement is a lot like a while loop

However, it only runs *at most once*, and only if the condition is true

unlike just plain move(), safe_move() will never result in a crash

SHORT ASSIGNMENT 1

- Do these exercises:



WARMUP:

- Make Karel put one beeper on every square at the edge of the world, then reverse direction and pick up the beeper. You must use at least one function other than `main()`, and at least one while loop

THE FUN STUFF:

- Make Karel draw a *checkerboard* pattern on the 9x9 world.

